

主讲老师: fox

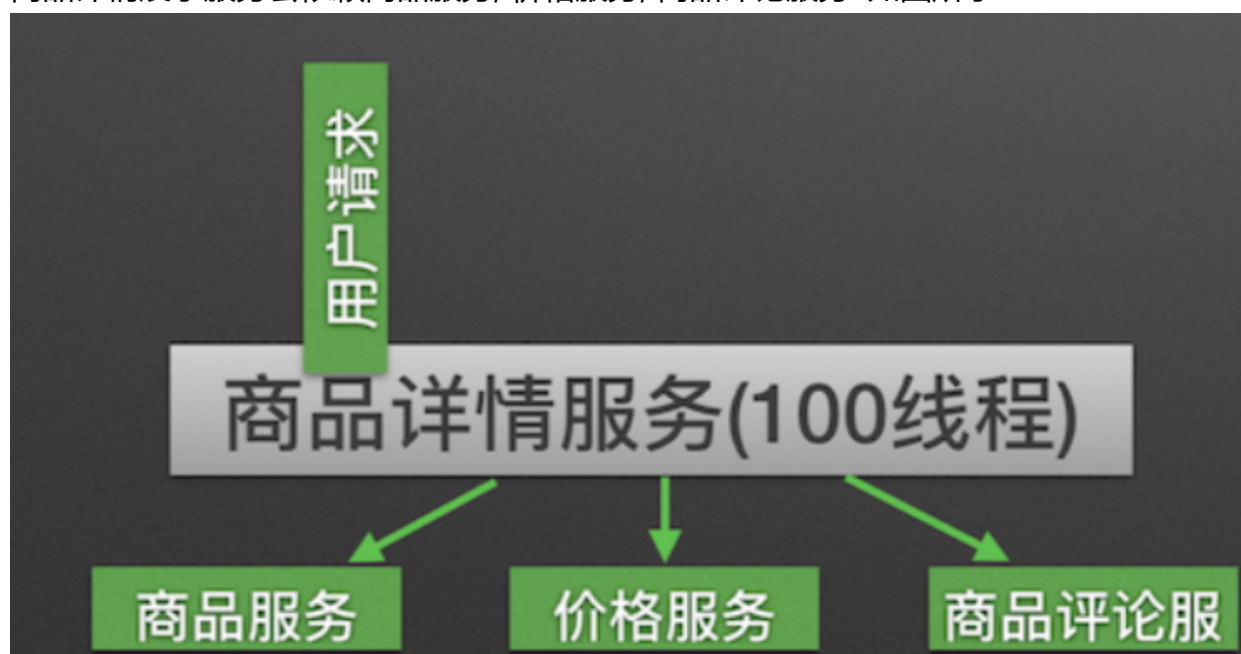
课程内容:

1. Hystrix使用和Feign的整合, Hystrix监控
2. 熔断器跳闸机制分析, 内部调用逻辑剖析
3. Hystrix源码分析

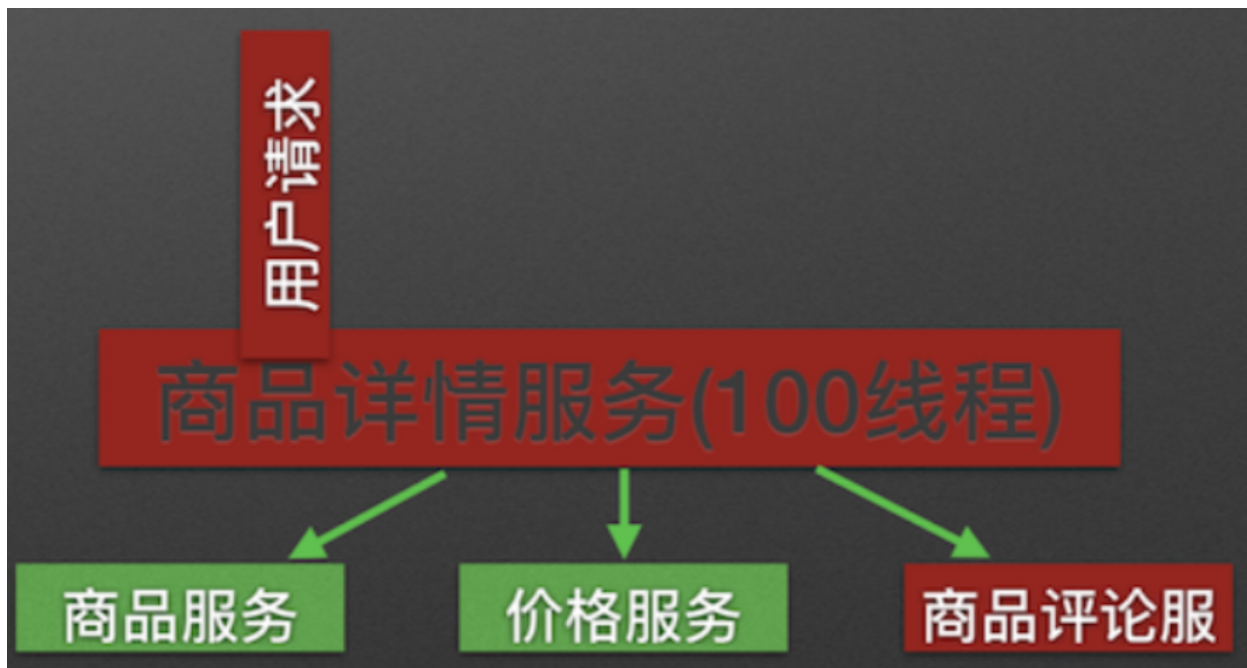
1. 前言

1.1) 分布式系统遇到的问题

在一个高度服务化的系统中,我们实现的一个业务逻辑通常会依赖多个服务,比如:商品详情展示服务会依赖商品服务, 价格服务, 商品评论服务. 如图所示:



调用三个依赖服务会共享商品详情服务的线程池. 如果其中的商品评论服务不可用, 就会出现线程池里所有线程都因等待响应而被阻塞, 从而造成**服务雪崩**. 如图所示:



服务雪崩效应：因服务提供者的不可用导致服务调用者的不可用,并将不可用逐渐放大的过程，就叫服务雪崩效应

导致服务不可用的原因： 程序Bug，大流量请求，硬件故障，缓存击穿

【大流量请求】：在秒杀和大促开始前,如果准备不充分,瞬间大量请求会造成服务提供者的不可用。

【硬件故障】：可能为硬件损坏造成的服务器主机宕机, 网络硬件故障造成的服务提供者的不可访问。

【缓存击穿】：一般发生在缓存应用重启, 缓存失效时高并发，所有缓存被清空时,以及短时间内大量缓存失效时。大量的缓存不命中, 使请求直击后端,造成服务提供者超负荷运行,引起服务不可用。

在服务提供者不可用的时候，会出现大量重试的情况：用户重试、代码逻辑重试，**这些重试最终导致：进一步加大请求流量**。所以归根结底导致雪崩效应的最根本原因是：**大量请求线程同步等待造成的资源耗尽**。当服务调用者使用同步调用时, 会产生大量的等待线程占用系统资源。一旦线程资源被耗尽,服务调用者提供的服务也将处于不可用状态, 于是服务雪崩效应产生了。

1.2) 解决方案

1. 超时机制
2. 服务限流(资源隔离)
3. 服务熔断
4. 服务降级

1.2.1) 超时机制

在不做任何处理的情况下，服务提供者不可用会导致消费者请求线程强制等待，而造成系统资源耗尽。加入超时机制，一旦超时，就释放资源。由于释放资源速度较快，一定程度上可以抑制资源耗尽的问题。

RestTemplate设置超时时间

```
1 @Bean
2 @LoadBalanced
3 public RestTemplate restTemplate() {
4     //设置restTemplate的超时时间
5     SimpleClientHttpRequestFactory requestFactory = new SimpleClientHttpRequestFactory();
6     requestFactory.setReadTimeout(2000);
7     requestFactory.setConnectTimeout(2000);
8     RestTemplate restTemplate = new RestTemplate(requestFactory);
9     return restTemplate;
10 }
11
12 @Bean
13 @LoadBalanced
14 public RestTemplate restTemplate() {
15     //设置restTemplate的超时时间
16     HttpComponentsClientHttpRequestFactory httpRequestFactory = new HttpComponentsClientHttpRequestFactory();
17     httpRequestFactory.setConnectionRequestTimeout(2000);
18     httpRequestFactory.setConnectTimeout(2000);
19     httpRequestFactory.setReadTimeout(2000);
20     return new RestTemplate(httpRequestFactory);
21 }
22
```

全局异常设置

```
1 @ControllerAdvice
2 public class MyExceptionHandler {
3
4     @ExceptionHandler(value = MyTimeoutException.class)
5     @ResponseBody
6     public Object handleException() {
7         UserInfoVo userInfoVo = new UserInfoVo();
8         userInfoVo.setUsername("超时异常");
9         userInfoVo.setOrderList(null);
10        return userInfoVo;
11    }
12 }
```

```

11  }
12  }
13  @Data
14  @AllArgsConstructor
15  public class MyTimeoutException extends RuntimeException {
16
17      private Integer code;
18
19      private String msg;
20  }

```

调用时捕获异常

```

1  String url = "http://service-order/order/findOrderByUserId/"+id;
2  ResponseEntity<List> responseEntity = null;
3  try {
4      responseEntity = restTemplate.getForEntity(url, List.class);
5  } catch (RestClientException e) {
6      throw new MyTimeoutException(-1, "调用超时");
7  }
8  List<Order> orderList = responseEntity.getBody();

```

1.2.2) 服务限流(资源隔离)

限制请求核心服务提供者的流量，使大流量拦截在核心服务之外，这样可以更好的保证核心服务提供者不出问题，对于一些出问题的服务可以限制流量访问，只分配固定线程资源访问，这样能使整体的资源不至于被出问题的服务耗尽，进而整个系统雪崩。那么服务之间怎么限流，怎么资源隔离？例如可以通过线程池+队列的方式，通过信号量的方式。

如下图所示，当商品评论服务不可用时，即使商品服务独立分配的20个线程全部处于同步等待状态，也不会影响其他依赖服务的调用。



1.1.3) 服务熔断

远程服务不稳定或网络抖动时暂时关闭，就叫服务熔断。

现实世界的断路器大家肯定都很了解，断路器实时监控电路的情况，如果发现电路电流异常，就会跳闸，从而防止电路被烧毁。

软件世界的断路器可以这样理解：实时监测应用，如果发现在一定时间内失败次数/失败率达到一定阈值，就“跳闸”，断路器打开——此时，请求直接返回，而不去调用原本调用的逻辑。跳闸一段时间后（例如10秒），断路器会进入半开状态，这是一个瞬间态，此时允许一次请求调用该调的逻辑，如果成功，则断路器关闭，应用正常调用；如果调用依然不成功，断路器继续回到打开状态，过段时间再进入半开状态尝试——通过“跳闸”，应用可以保护自己，而且避免浪费资源；而通过半开的设计，可实现应用的“自我修复”。

所以，同样的道理，当依赖的服务有大量超时，在让新的请求去访问根本没有意义，只会无畏的消耗现有资源。比如我们设置了超时时间为1s,如果短时间内有大量请求在1s内都得不到响应，就意味着这个服务出现了异常，此时就没有必要再让其他的请求去访问这个依赖了，这个时候就应该使用断路器避免资源浪费。

1.1.4) 服务降级

有服务熔断，必然要有服务降级。

所谓降级，就是当某个服务熔断之后，服务将不再被调用，此时客户端可以自己准备一个本地的fallback（回退）回调，返回一个缺省值。例如：（备用接口/缓存/mock数据）。这样做，虽然服务水平下降，但好歹可用，比直接挂掉要强，当然这也要看适合的业务场景。

2. Hystrix快速开始

Hystrix（豪猪）是由Netflix开源的一个延迟和容错库，提供**超时机制，限流，熔断，降级**最全面的实现，用于隔离访问远程系统、服务或者第三方库，防止级联失败，从而提升系统的可用性与容错性。

2.1) 引入依赖

```
1 <dependency>
2   <groupId>org.springframework.cloud</groupId>
3   <artifactId>spring-cloud-starter-netflix-hystrix</artifactId>
4 </dependency>
```

2.2) 启动类加@EnableCircuitBreaker注解

```
1 @SpringBootApplication
2 @EnableCircuitBreaker
3 public class HystrixConsumerApplication {
4
5     public static void main(String[] args) {
6         SpringApplication.run(HystrixConsumerApplication.class, args);
7     }
8 }
```

```
7    }  
8    }
```

2.3) 服务降级方式

2.3.1) 自定义命令行调用方式

命令模式的方式：继承(**HystrixCommand**类)来包裹具体的服务调用逻辑(run方法), 并在命令模式中添加了服务调用失败后的降级逻辑(**getFallback**)

```
1  @Slf4j  
2  public class MyHystrixCommand extends HystrixCommand<List<Order>> {  
3  
4      private Integer userId;  
5  
6      private RestTemplate restTemplate;  
7  
8      public MyHystrixCommand(String commandGroupKey, Integer userId, RestTemplate restTemplate) {  
9          super(HystrixCommandGroupKey.Factory.asKey(commandGroupKey));  
10         this.userId = userId;  
11         this.restTemplate = restTemplate;  
12     }  
13  
14     @Override  
15     protected List<Order> run() throws Exception {  
16         String url = "http://service-order/order/findOrderByUserId/"+userId;  
17         ResponseEntity<List> responseEntity =  
18             restTemplate.getForEntity(url, List.class);  
19         List<Order> orderList = responseEntity.getBody();  
20         log.info("查询orderList:"+ orderList);  
21         return orderList;  
22     }  
23  
24     @Override  
25     protected List<Order> getFallback() {  
26         log.info("===调用降级方法===");  
27         Order order = new Order();  
28         order.setId(-1);  
29         order.setUserId(-1);  
30         order.setAmount(0);  
31  
32         List<Order> orderList = new ArrayList<>();  
33         orderList.add(order);
```

```

34     return orderList;
35 }
36 }
37
38 //UserController.java
39
40 @RequestMapping(value = "/getById/{id}")
41 public UserInfoVo getUserById(@PathVariable("id") Integer id) {
42     User user = userService.getById(id);
43
44     //调用命令模式
45     MyHystrixCommand myHystrixCommand =
46     new MyHystrixCommand("orderGroupKey",id,restTemplate);
47     List<Order> orderList =myHystrixCommand.execute();
48
49     UserInfoVo userInfoVo = new UserInfoVo();
50     userInfoVo.setOrderList(orderList);
51     userInfoVo.setUsername(user.getUsername());
52
53     return userInfoVo;
54 }

```

2.3.2) 通过@HystrixCommand指定降级方法

```

1  @HystrixCommand(fallbackMethod ="queryUserInfoFallBack")
2  @RequestMapping("/getById/{id}")
3  public UserInfoVo getUserById(@PathVariable("id") Integer id) {
4      User user = userService.getById(id);
5
6      String url = "http://service-order/order/findOrderByUserId/"+id;
7      ResponseEntity<List> responseEntity =
8      restTemplate.getForEntity(url, List.class);
9      List<Order> orderList = responseEntity.getBody();
10
11     UserInfoVo userInfoVo = new UserInfoVo();
12     userInfoVo.setOrderList(orderList);
13     userInfoVo.setUsername(user.getUsername());
14
15     return userInfoVo;
16 }

```


测试：服务宕机，超时，服务异常

```
1 @RequestMapping("/findOrderByUserId/{userId}")
2 public List<Order> findOrderByUserId(@PathVariable("userId") Integer userId) {
3     // try {
4     // // 测试熔断，模拟超时
5     // Thread.sleep(2000);
6     // } catch (InterruptedException e) {
7     // e.printStackTrace();
8     // }
9
10    //测试熔断，模拟服务异常
11    if(userId == 2){
12        throw new NullPointerException();
13    }
14    List<Order> list = orderService.findOrderByUserId(userId);
15    return list;
16 }
```

2.4) 限流，线程隔离实现

```
1 @RequestMapping("/findOrderByUserId/{userId}")
2 public List<Order> findOrderByUserId(@PathVariable("userId") Integer userId) {
3     //模拟限流，线程资源隔离，模拟系统执行速度很慢的情况
4     //修改consumer端hystrix超时配置为20s
5     try {
6         Thread.sleep(3000);
7     } catch (InterruptedException e) {
8         e.printStackTrace();
9     }
10
11    List<Order> list = orderService.findOrderByUserId(userId);
12    return list;
13 }
```

修改调用者超时时间

```
1 #设置接口的超时时间（默认1秒）
2 hystrix.command.default.execution.isolation.thread.timeoutInMilliseconds=
0000
```

然后用注解配置线程池大小：

```
1 @HystrixCommand(fallbackMethod ="queryUserInfoFallBack",
```



```

2  groupKey = "orderGroup",threadPoolKey = "orderThreadPool",
3  threadPoolProperties = {
4  @HystrixProperty(name="coreSize",value="2"),
5  // @HystrixProperty(name="maxQueueSize",value="5"),
6  // @HystrixProperty(name="queueSizeRejectionThreshold",value="3")
7  }
8  )

```

CommandGroupKey: 配置全局唯一标识服务分组的名称，比如，库存系统就是一个服务分组。当我们监控时，相同分组的服务会聚合在一起，必填选项。

CommandKey: 配置全局唯一标识服务的名称，比如，库存系统有一个获取库存服务，那么就可以为这个服务起一个名字来唯一识别该服务，如果不配置，则默认是简单类名。

ThreadPoolKey: 配置全局唯一标识线程池的名称，相同线程池名称的线程池是同一个，如果不配置，则默认是分组名，此名字也是线程池中线程名字的前缀。

ThreadPoolProperties: 配置线程池参数，coreSize配置核心线程池大小和线程池最大大小，keepAliveTimeMinutes是线程池中空闲线程生存时间（如果不进行动态配置，那么是没有任何作用的），maxQueueSize配置线程池队列最大大小，queueSizeRejectionThreshold限定当前队列大小，即实际队列大小由这个参数决定，通过改变queueSizeRejectionThreshold可以实现动态队列大小调整。

CommandProperties: 配置该命令的一些参数，如executionIsolationStrategy配置执行隔离策略，默认是使用线程隔离，此处我们配置为THREAD，即线程池隔离。

此处可以粗粒度实现隔离，也可以细粒度实现隔离，如下所示：

服务分组+线程池：粗粒度实现，一个服务分组/系统配置一个隔离线程池即可，不配置线程池名称或者相同分组的线程池名称配置为一样。

服务分组+服务+线程池：细粒度实现，一个服务分组中的每一个服务配置一个隔离线程池，为不同的命令实现配置不同的线程池名称即可。

混合实现：一个服务分组配置一个隔离线程池，然后对重要服务单独设置隔离线程池。

基于信号量：当 **计数器 + 1 > 设置的最高并发数** 时，就限流，或者走降级渠道

2.5) 属性配置

ribbon+hystrix超时问题：开启超时配置

```

1  #设置接口的超时时间（默认1秒） 参考： HystrixCommandProperties
2  hystrix.command.default.execution.isolation.thread.timeoutInMilliseconds=
  000

```

```
3 #指定特定接口的超时时间
4 hystrix.command.queryUserInfoFallback.execution.isolation.thread
5 .timeoutInMilliseconds=3000
```

2.6) hystrix整合feign

启动类上配置feign的注解

```
1 @SpringBootApplication
2 @EnableCircuitBreaker
3 @EnableFeignClients(basePackages = "bat.ke.qq.com")
4 public class FeignHystrixConsumerApplication {
5
6     public static void main(String[] args) {
7         SpringApplication.run(FeignHystrixConsumerApplication.class,args);
8     }
9 }
```

修改配置文件

```
1 #设置接口的超时时间（默认1秒）
2 hystrix.command.default.execution.isolation.thread.timeoutInMilliseconds=
3 000
4 #开启feign支持hystrix 默认是关闭的
5 feign.hystrix.enabled=true
6
7 # ribbon全局超时时间设置（使用feign需要设置）
8 ribbon.ReadTimeout=8000
9 ribbon.ConnectTimeout=2000
```

2.7) 监控

2.7.1) 访问hystrix.stream的监控端点

引入依赖

```
1 <dependency>
2 <groupId>org.springframework.boot</groupId>
3 <artifactId>spring-boot-starter-actuator</artifactId>
4 </dependency>
```

加入监控端点配置

```
1 @Bean
2 public ServletRegistrationBean getServlet(){
3     HystrixMetricsStreamServlet streamServlet = new HystrixMetricsStreamServ
4     let();
```

```

4 ServletRegistrationBean registrationBean = new ServletRegistrationBean(s
treamServlet);
5 registrationBean.setLoadOnStartup(1);
6 registrationBean.addUrlMappings("/hystrix.stream");
7 registrationBean.setName("HystrixMetricsStreamServlet");
8 return registrationBean;
9 }

```

测试，访问端点： <http://localhost:8200/hystrix.stream>

2.7.2) Hystrix+dashboard监控

引入依赖

```

1 <dependency>
2   <groupId>org.springframework.cloud</groupId>
3   <artifactId>spring-cloud-starter-netflix-hystrix-dashboard</artifactId>
4 </dependency>

```

启动类上添加@EnableHystrixDashboard注解，开启仪表盘功能

```

1 @SpringBootApplication
2 @EnableHystrixDashboard
3 public class ServiceHystrixDashboardApplication {
4
5   public static void main(String[] args) {
6     SpringApplication.run(ServiceHystrixDashboardApplication.class, args);
7   }
8
9 }

```

配置属性

```

1 server.port=10000
2 spring.application.name=hystrix-dashboard

```

测试： <http://localhost:10000/hystrix>

输入<http://localhost:8200/hystrix.stream>



Hystrix Dashboard

`http://hostname:port/turbine/turbine.stream` ← 这里输入要监控的服务地址

Cluster via Turbine (default cluster): `http://turbine-hostname:port/turbine.stream`

Cluster via Turbine (custom cluster): `http://turbine-hostname:port/turbine.stream?cluster=[clusterName]`

Single Hystrix App: `http://hystrix-app:port/hystrix.stream`

Delay: ms

Title:

轮询监控的延迟时间，默认2000ms

Example Hystrix App

Monitor Stream

仪表盘上的标题，默认使用URL

实心圆的颜色从绿黄橙红依次递减，表示服务的健康程度依次降低，圆越大，表示服务的流量越大

Hystrix Stream: myconsumer



2.7.3) turbine集群监控

引入依赖

```
1 <dependency>
2   <groupId>org.springframework.cloud</groupId>
3   <artifactId>spring-cloud-starter-netflix-turbine</artifactId>
4 </dependency>
```

启动类上配置@EnableTurbine注解

```
1 @SpringBootApplication
```

```

2 @EnableTurbine
3 public class ServiceTurbineApplication {
4
5     public static void main(String[] args) {
6         SpringApplication.run(ServiceTurbineApplication.class, args);
7     }
8
9 }

```

配置属性

```

1 server.port=11000
2
3 #注册到eureka服务端的微服务名称
4 spring.application.name=service-turbine
5
6 #暴露给其他eureka client 的注册地址
7 eureka.client.service-url.defaultZone=http://www.eureka8761.com:8761/eureka/
8 #将ip注册到Eureka Server上
9 eureka.instance.prefer-ip-address=true
10 #显示微服务的服务实例id
11 eureka.instance.instance-id=${spring.application.name}-${server.port}
12
13 # 指定聚合哪些集群，多个使用","分割，默认为default。
14 # 可使用http://.../turbine.stream?cluster={clusterConfig之一}访问
15 turbine.aggregator.cluster-config=default
16 #指定要监控的微服务名
17 turbine.app-config=service-hystrix-consumer,service-feign-hystrix-consumer
18 #集群的名字为default
19 turbine.cluster-name-expression="default"
20 #同一主机上的服务通过host和port的组合来进行区分
21 turbine.combine-host-port=true
22 # 默认path: /actuator/hystrix.stream
23 turbine.instanceUrlSuffix= /hystrix.stream

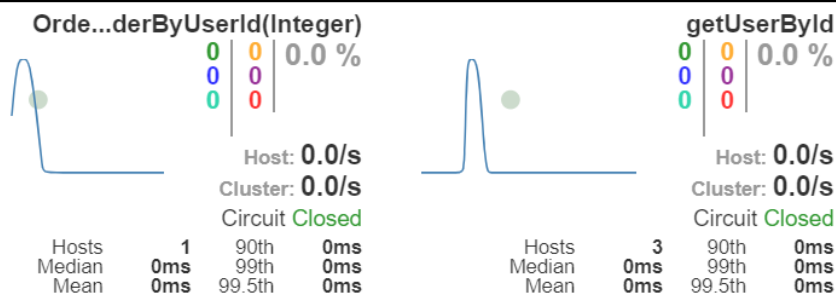
```

测试 <http://localhost:10000/hystrix>

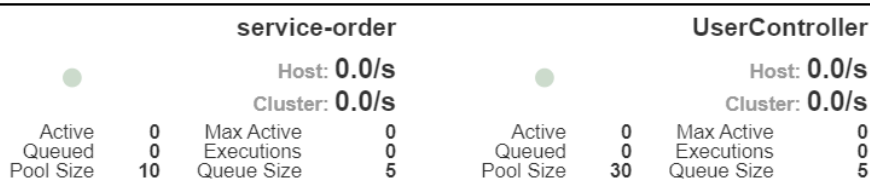
输入<http://localhost:11000/turbine.stream>

Hystrix Stream: http://localhost:11000/turbine.stream

Circuit Sort: [Error then Volume](#) | [Alphabetical](#) | [Volume](#) | [Error](#) | [Mean](#) | [Median](#) | [90](#) | [99](#) | [99.](#)

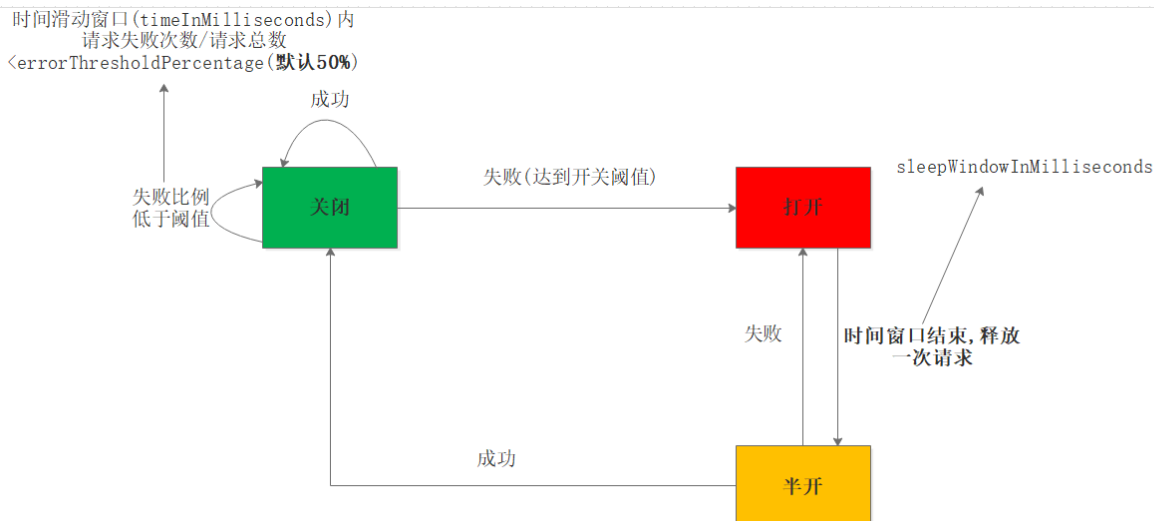


Thread Pools Sort: [Alphabetical](#) | [Volume](#) |



3. Hystrix原理

3.1) 熔断器跳闸机制三态转换图



`hystrix.command.default.circuitBreaker.requestVolumeThreshold` 一个rolling window内最小的请求数。如果设为20, 那么当一个rolling window的时间内 (比如说1个rolling window是10秒) 收到19个请求, 即使19个请求都失败, 也不会触发circuit break。默认20

`hystrix.command.default.circuitBreaker.sleepWindowInMilliseconds` 触发短路的时间值，当该值设为5000时，则当触发circuit break后的5000毫秒内都会拒绝request，也就是5000毫秒后才会关闭circuit。默认5000

`hystrix.command.default.metrics.rollingStats.timeInMilliseconds` 设置统计的时间窗值，毫秒值，circuit break 的打开会根据1个rolling window的统计来计算。若rolling window被设为10000毫秒，则rolling window会被分成n个buckets，每个bucket包含success, failure, timeout, rejection的次数的统计信息。默认10000

3.2) 测试熔断打开以及半开

```
1 # 查看详细的健康检查信息
2 management.endpoint.health.show-details=always
```

访问: <http://localhost:8200/actuator/health>

查看监控端点hystrix的值:

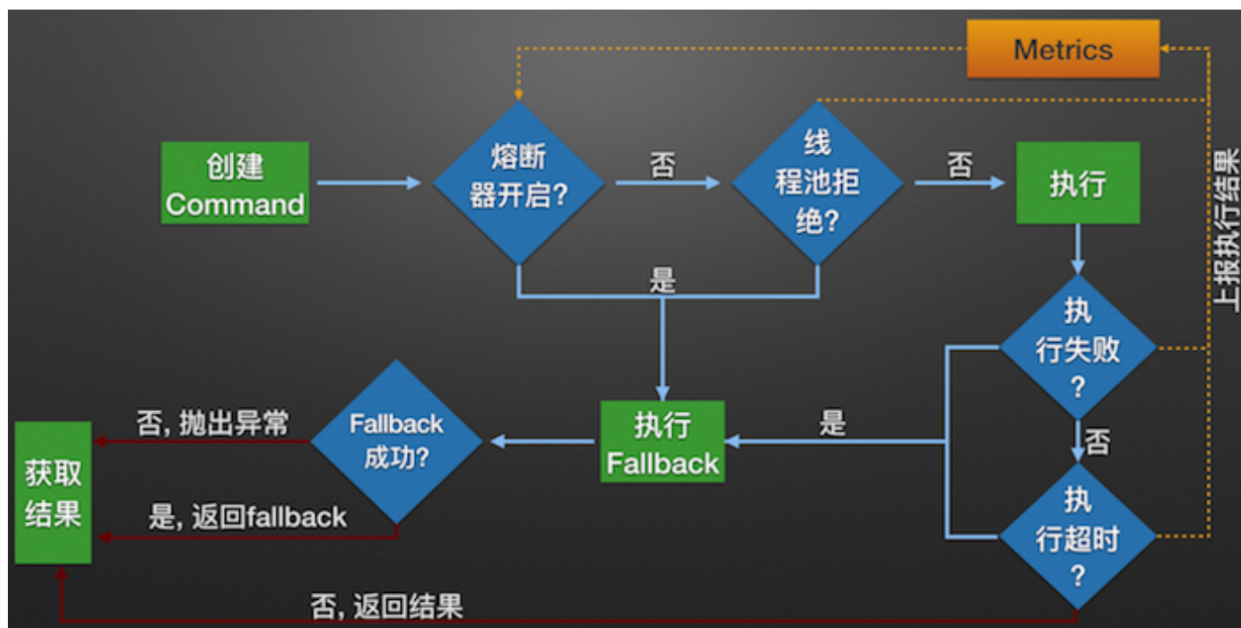
```
- hystrix: {
  status: "UP"
}
```

在时间窗口内，连续点击失败次数,当此时达到设置的requestVolumeThreshold的阈值，就直接进入降级方法,此时再来看hystrix的监控信息:

```
- hystrix: {
  status: "CIRCUIT_OPEN",
  - details: {
    - openCircuitBreakers: [
      "UserController::getUserById"
    ]
  }
}
```

等到熔断器半开后，测试一个正确的查询，那么熔断器就会关闭，恢复正常调用

3.2 Hystrix服务调用的内部逻辑



- 1.构建Hystrix的Command对象, 调用执行方法.
- 2.Hystrix检查当前服务的熔断器开关是否开启, 若开启, 则执行降级服务getFallback方法.
- 3.若熔断器开关关闭, 则Hystrix检查当前服务的线程池是否能接收新的请求, 若线程池已满, 则执行降级服务getFallback方法.
- 4.若线程池接受请求, 则Hystrix开始执行服务调用具体逻辑run方法.
- 5.若服务执行失败, 则执行降级服务getFallback方法, 并将执行结果上报Metrics更新服务健康状况.
- 6.若服务执行超时, 则执行降级服务getFallback方法, 并将执行结果上报Metrics更新服务健康状况.
- 7.若服务执行成功, 返回正常结果.
- 8.若服务降级方法getFallback执行成功, 则返回降级结果.
- 9.若服务降级方法getFallback执行失败, 则抛出异常.

Hystrix Metrics的实现

Hystrix的Metrics中保存了当前服务的健康状况, 包括服务调用总次数和服务调用失败次数等. 根据Metrics的计数, 熔断器从而能计算出当前服务的调用失败率, 用来和设定的阈值比较从而决定熔断器的状态切换逻辑.

3.3 Hystrix相关配置

hystrix.command.default和hystrix.threadpool.default中的default为默认

CommandKey

Command Properties

3.3.1) Execution相关的属性的配置:

1. `hystrix.command.default.execution.isolation.strategy` 隔离策略，默认是 Thread, 可选 Thread | Semaphore
2. `hystrix.command.default.execution.isolation.thread.timeoutInMilliseconds` 命令执行超时时
间，默认1000ms
3. `hystrix.command.default.execution.timeout.enabled` 执行是否启用超时，默认
启用true
4. `hystrix.command.default.execution.isolation.thread.interruptOnTimeout` 发
生超时是是否中断，默认true
5. `hystrix.command.default.execution.isolation.semaphore.maxConcurrentRequests` 最大并发请求数，默认10，该参数当使用
ExecutionIsolationStrategy.SEMAPHORE策略时才有效。如果达到最大并发请求
数，请求会被拒绝。理论上选择semaphore size的原则和选择thread size一致，但
选用semaphore时每次执行的单元要比较小且执行速度快（ms级别），否则的话应
该thread。semaphore应该占整个容器（tomcat）的线程池的一小部分。

3.3.2) Fallback相关的属性

这些参数可以应用于Hystrix的THREAD和SEMAPHORE策略

1. `hystrix.command.default.fallback.isolation.semaphore.maxConcurrentRequests` 如果并发数达到该设置值，请求会被拒绝和抛出异常并且fallback不会被调用。默
认10
2. `hystrix.command.default.fallback.enabled` 当执行失败或者请求被拒绝，是否
会尝试调用
`hystrixCommand.getFallback()` 。默认true

3.3.3) Circuit Breaker相关的属性

1. `hystrix.command.default.circuitBreaker.enabled` 用来跟踪circuit的健康性，如
果未达标则让request短路。默认true
2. `hystrix.command.default.circuitBreaker.requestVolumeThreshold` 一个
rolling window内最小的请求数。如果设为20，那么当一个rolling window的时间
内（比如说1个rolling window是10秒）收到19个请求，即使19个请求都失败，也
不会触发circuit break。默认20

3. `hystrix.command.default.circuitBreaker.sleepWindowInMilliseconds` 触发短路的时间值，当该值设为5000时，则当触发circuit break后的5000毫秒内都会拒绝request，也就是5000毫秒后才会关闭circuit。默认5000
4. `hystrix.command.default.circuitBreaker.errorThresholdPercentage` 错误比率阈值，如果错误率 $\geq$ 该值，circuit会被打开，并短路所有请求触发fallback。默认50
5. `hystrix.command.default.circuitBreaker.forceOpen` 强制打开熔断器，如果打开这个开关，那么拒绝所有request，默认false
6. `hystrix.command.default.circuitBreaker.forceClosed` 强制关闭熔断器 如果这个开关打开，circuit将一直关闭且忽略`circuitBreaker.errorThresholdPercentage`

3.3.4) Metrics相关参数

1. `hystrix.command.default.metrics.rollingStats.timeInMilliseconds` 设置统计的时间窗口值的，毫秒值，circuit `break` 的打开会根据1个rolling window的统计来计算。若rolling window被设为10000毫秒，则rolling window会被分成n个buckets，每个bucket包含success, failure, timeout, rejection的次数的统计信息。默认10000
2. `hystrix.command.default.metrics.rollingStats.numBuckets` 设置一个rolling window被划分的数量，若numBuckets = 10, rolling window = 10000，那么一个bucket的时间即1秒。必须符合 $\text{rolling window} \% \text{numberBuckets} == 0$ 。默认10
3. `hystrix.command.default.metrics.rollingPercentile.enabled` 执行时是否enable指标的计算和跟踪，默认true
4. `hystrix.command.default.metrics.rollingPercentile.timeInMilliseconds` 设置rolling percentile window的时间，默认60000
5. `hystrix.command.default.metrics.rollingPercentile.numBuckets` 设置rolling percentile window的numberBuckets。逻辑同上。默认6
6. `hystrix.command.default.metrics.rollingPercentile.bucketSize` 如果bucket size = 100, window = 10s，若这10s里有500次执行，只有最后100次执行会被统计到bucket里去。增加该值会增加内存开销以及排序的开销。默认100
7. `hystrix.command.default.metrics.healthSnapshot.intervalInMilliseconds` 记录health 快照（用来统计成功和错误绿）的间隔，默认500ms

3.3.5) Request Context 相关参数

1. `hystrix.command.default.requestCache.enabled` 默认true, 需要重载 `getCacheKey()`, 返回null时不缓存
2. `hystrix.command.default.requestLog.enabled` 记录日志到 `HystrixRequestLog`, 默认true

3.3.6) Collapser Properties 相关参数

1. `hystrix.collapse.default.maxRequestsInBatch` 单次批处理的最大请求数, 达到该数量触发批处理, 默认 `Integer.MAX_VALUE`
2. `hystrix.collapse.default.timerDelayInMilliseconds` 触发批处理的延迟, 也可以为创建批处理的时间 + 该值, 默认10
3. `hystrix.collapse.default.requestCache.enabled` 是否对 `HystrixCollapse.execute()` and `HystrixCollapse.queue()`的cache, 默认true

3.3.7) ThreadPool 相关参数

1. 线程数默认值10, 适用于大部分情况 (有时可以设置得更小), 如果需要设置得更大, 那有个基本的公式可以follow: $\text{requests per second at peak when healthy} \times 99\text{th percentile latency in seconds} + \text{some breathing room}$ 每秒最大支撑的请求数 (99%平均响应时间 + 缓存值)

比如: 每秒能处理1000个请求, 99%的请求响应时间是60ms, 那么公式是:

1000 (0.060+0.012) 基本的原则是保持线程池尽可能小, 主要是为了释放压力, 防止资源被阻

塞。当一切都是正常的时候, 线程池一般仅会有1到2个线程激活来提供服务。

2. `hystrix.threadpool.default.coreSize` 并发执行的最大线程数, 默认10

3. `hystrix.threadpool.default.maxQueueSize` `BlockingQueue`的最大队列数, 当设为 - 1, 会使用

`SynchronousQueue`, 值为正时使用 `LinkedBlockingQueue`。该设置只会在初始化时有效, 之后

不能修改threadpool的queue size, 除非reinitialising thread executor。默认 - 1。

4. `hystrix.threadpool.default.queueSizeRejectionThreshold` 即使 `maxQueueSize` 没有达到, 达到

`queueSizeRejectionThreshold`该值后, 请求也会被拒绝。因为 `maxQueueSize` 不能被动态修改,

这个参数将允许我们动态设置该值。if `maxQueueSize == 1`, 该字段将不起作用

5. `hystrix.threadpool.default.keepAliveTimeMinutes` 如果`corePoolSize`和`maxPoolSize`设成一样（默认实现）该设置无效。如果通过plugin (<https://github.com/Netflix/Hystrix/wiki/Plugins>) 使用自定义实现，该设置才有用，默认1.
6. `hystrix.threadpool.default.metrics.rollingStats.timeInMilliseconds` 线程池统计指标的时间，默认10000
7. `hystrix.threadpool.default.metrics.rollingStats.numBuckets` 将rolling window划分为n个buckets，默认10

4. Hystrix源码分析

源码分析: <https://www.processon.com/view/link/5e81f9a6e4b0984efb8126c9>